



Project Title: Pharmacy Inventory Management System + Cloud Analytics Export Pipeline

Prepared By: *(Your Name) | (Roll*

*No) | (Course/Section)***Instructor:** *(Name)***Institution:** *(Name)***Date:** *(DD/MM/YYYY)*

Abstract

This report presents a full-stack Pharmacy Inventory Management System that supports secure authentication, inventory control, order processing, supplier management, and dashboard analytics. The system stores operational data in MongoDB Atlas and provides an export pipeline using AWS Lambda to store analytics-ready datasets in Amazon S3. AWS Glue Crawlers catalog S3 exports into the Glue Data Catalog, enabling SQL-based analysis via Amazon Athena. The implementation follows cloud security best practices including least-privilege access, VPC isolation for Lambda, encryption at rest in S3, and centralized logging through CloudWatch.

Table of Contents

List of Figures (LOF)

- Figure 1:** Overall Architecture Diagram (Operational Flow + Analytics Flow)
- Figure 2:** VPC Design (Public/Private Subnets + IGW + NAT)
- Figure 3:** Lambda Export Function Configuration (VPC + Env Vars)
- Figure 4:** S3 Export Folder Structure (exports/<collection>/)
- Figure 5:** Glue Database and Crawlers Configuration
- Figure 6:** Glue Data Catalog Tables Created
- Figure 7:** Athena Settings (Query Result Location)
- Figure 8:** Athena Query Results on Exported Data
- Figure 9:** CloudWatch Logs for Lambda Export Execution
- Figure 10:** EC2 Backend Health Check and Running Service (if EC2 used)
-

List of Tables (LOT)

Table 1: Technology Stack Summary

Layer	Technology/Service	Purpose in Project
Frontend	React 19	Web UI for pharmacy operations
Frontend UI	Material UI (MUI)	Component library + theming (light/dark)
Frontend Routing	React Router	Client-side navigation
Frontend HTTP	Axios	API requests to backend
Frontend State	React Context API	Auth, theme, notifications
Backend Runtime	Node.js	Server runtime for API

Layer	Technology/Service	Purpose in Project
Backend Framework	Express.js	REST API endpoints
Auth	JWT (jsonwebtoken)	Secure login + protected routes
Password Hashing	bcryptjs	Hash and verify passwords
Database	MongoDB Atlas	Primary operational database
Cross-Origin	CORS	Allow frontend domain to call backend
Hosting (Frontend)	AWS Amplify	Deploy React app
Hosting (Backend)	EC2	Hosts Node/Express API server
User Data Script	Boots EC2 instances with latest code	Installs Node, Git, PM2, CodeDeploy agent, clones repo, sets up .env
Data Export	AWS Lambda (Node.js 20)	Export MongoDB collections to S3
Storage	Amazon S3	Store NDJSON exports + Athena query results
Networking	VPC + Private Subnets + NAT Gateway	Private Lambda + stable outbound IP for Atlas allowlist
Catalog	AWS Glue (Crawler + Data Catalog)	Infer schema and create tables from S3
Query Engine	Amazon Athena	SQL queries over S3 datasets

Layer	Technology/Service	Purpose in Project
Monitoring	CloudWatch Logs	Logs for Lambda executions
Version Control	Git/GitHub	Source control and deployment triggers

Table 2: Core API Endpoints and Access Control

Module	Endpoint	Method	Access	Purpose
Health	/api/health	GET	Public	Verify API is running
Auth	/api/auth/register	POST	Public	Register user
Auth	/api/auth/login	POST	Public	Login user (returns JWT)
Auth	/api/auth/me	GET	Protected	Get current user profile
Auth	/api/auth/profile	PUT	Protected	Update profile
Auth	/api/auth/password	PUT	Protected	Change password
Admin Users	/api/auth/users	GET	Admin	List users
Admin Users	/api/auth/users/:id/role	PUT	Admin	Update user role
Admin Users	/api/auth/users/:id/status	PUT	Admin	Activate/deactivate user
Products	/api/products	GET	Protected	List products (filters/pagination)

Module	Endpoint	Method	Access	Purpose
Products	/api/products/:id	GET	Protect ed	Get product by ID
Products	/api/products	POST	Admin	Create product
Products	/api/products/:id	PUT	Admin	Update product
Products	/api/products/:id	DELETE	Admin	Soft-delete product
Stock	/api/products/:id/st ock	PATCH	Protect ed	Update stock (add/subtract)
Orders	/api/orders	GET	Protect ed	List orders
Orders	/api/orders	POST	Protect ed	Create order
Orders	/api/orders/:id	GET	Protect ed	Get order by ID
Orders	/api/orders/:id	PUT	Protect ed	Update order
Orders	/api/orders/:id/stat us	PUT/PAT CH	Protect ed	Update order status
Orders	/api/orders/:id/pay ment	PUT	Protect ed	Update payment status
Orders	/api/orders/:id	DELETE	Admin	Delete order (restores stock if needed)
Suppliers	/api/suppliers	GET	Protect ed	List suppliers

Module	Endpoint	Method	Access	Purpose
Suppliers	/api/suppliers	POST	Admin	Create supplier
Suppliers	/api/suppliers/:id	PUT	Admin	Update supplier
Suppliers	/api/suppliers/:id	DELETE	Admin	Delete supplier
Dashboard	/api/dashboard/stats	GET	Protected	KPIs for dashboard
Reports	/api/reports/dashboard	GET	Protected	Dashboard report data
Reports	/api/reports/sales	GET	Protected	Sales report
Reports	/api/reports/inventory	GET	Protected	Inventory report

Table 3: Database Collections and Key Fields

Database	Collection	Key Fields (Examples)	Notes
test	users	name, email, password(hash), role, isActive, lastLogin	RBAC: admin/manager/user
test	products	productName, category, quantity, threshold, expiryDate, supplier, buyingPrice, sellingPrice	Stock and expiry tracking
test	orders	orderNumber, customer{name,phone,email}, items[{product,quantity,unitPrice,totalPrice}], total, paymentStatus, orderStatus, createdBy	Stock sync on confirmed/canceled/delete

Dat abase	Coll ectio n	Key Fields (Examples)	Notes
test	supp liers	name, email, phone, <i>(other supplier fields)</i>	Linked to products via sup plier ref

Table 4: AWS Services Used and Justification

AWS Se rvice	Used For	Why This Service (Justificatio n)	Security/Best Practice Applied
AWS A mplify	Host React fronten d	Fast static hosting + Git- based deployments	HTTPS by default; env vars for API URL
Amazon EC2	Host Node/Express backend	Persistent server runtime f or API	Security groups restrict in bound/outbound
AWS La mbda	Export job (MongoDB → S3)	Serverless batch export, runs on-demand	Runs in private subnets; logs to CloudWatch
Amazon S3	Store exports + Athena results	Durable, low- cost object storage	Block public access ON; SSE-S3 encryption
AWS Gl ue	Catalog S3 datasets	Automated schema infere nce + Data Catalog tables	IAM-controlled access; crawler scoped to prefixe s
Amazon Athena	Query S3 data using SQL	Serverless analytics without managing DB	Query results stored to restricted S3 path
Amazon VPC	Network isolation	Private subnets for Lamb da + controlled routing	Subnet separation (public/private)

AWS Service	Used For	Why This Service (Justification)	Security/Best Practice Applied
NAT Gateway	Outbound internet for private Lambda	Stable egress IP for Atlas allowlisting	Only used for outbound; private route to NAT
Internet Gateway	Internet access for public subnet/NAT	Required for NAT to reach internet	Standard VPC networking
CloudWatch Logs	Logging for Lambda	Debugging + monitoring export runs	Centralized logs; evidence for testing

Table 5: IAM Constraints and Workarounds (AWS Academy/Lab)

IAM Constraint (Observed)	Example/Error	Impact on Project	Workaround Implemented	Security Note
Cannot create IAM roles	iam:CreateRole not allowed	Could not create a custom Lambda execution role	Used existing Lambda execution role (LabRole) provided by the lab	Keep least privilege by scoping S3 access tightly
Limited ability to attach/modify IAM policies	Role/policy attach options restricted	Hard to grant S3 permissions via IAM policy	Used S3 Bucket Policy to allow the Lambda role s3:PutObject	Bucket policy scoped to arn:aws:s3:::<bucket>/exports/* only

IAM Constraint (Observed)	Example/Error	Impact on Project	Workaround Implemented	Security Note
iam:Pass Role restrictions	Unable to select new roles for services	Blocked creating/assigning new service roles (Glue/Lambda)	Selected existing allowed roles during service setup	Verify the selected role has only required permissions
Restricted visibility of IAM resources	Some roles/policies not listed	Hard to choose correct roles for Glue crawlers	Used the roles visible in console and validated by running crawlers/tests	Confirm success via Glue run status + CloudWatch logs
Secrets management limitations	Cannot set up new secret policies/roles easily	Harder to centralize secrets in Secrets Manager	Used Lambda environment variables for MONGODB_URI and kept them out of Git	Rotate exposed DB credentials and avoid screenshots showing secrets
Over-permissive defaults risk	Lab roles may be broader than ideal	Potentially more access than needed	Compensated by tightening resource policies (S3 bucket policy) and network allowlists	Use Atlas IP allowlisting + S3 block public access + encryption

2. Introduction

Pharmacies require accurate inventory control to prevent stock-outs, expired product losses, and incorrect order fulfillment. Manual processes and fragmented tools reduce operational efficiency and introduce errors. This project provides a unified web-based platform for pharmacy operations with secure role-based access and a separate cloud analytics pipeline to export and query data efficiently.

3. Problem Statement

- Manual inventory tracking causes inaccurate quantities and delayed restocking.
 - Order creation and deletion can desynchronize inventory without workflow enforcement.
 - Operational data becomes difficult to analyze without a query able analytics layer.
 - Security and access control are required to restrict sensitive actions to administrators.
-

4. Aim & Objectives

Aim

To build a secure pharmacy management system with an AWS-based export pipeline enabling analytics via Athena.

Objectives

- Implement JWT authentication and RBAC (Admin/Manager/User).
 - Provide modules: Inventory, Orders, Suppliers, Dashboard, Admin Users.
 - Ensure DB-consistent restock and order-based inventory synchronization.
 - Export MongoDB Atlas data to S3 via Lambda and enable querying using Glue + Athena.
 - Provide monitoring logs and evidence of outputs.
-

5. Scope of Work

In-Scope

- Full-stack pharmacy operations platform
- Cloud deployment (Frontend + Backend)
- Handle Traffic through Load Balancer
- Export pipeline: MongoDB → Lambda → S3 → Glue → Athena
- Logging and monitoring

Out-of-Scope

- Full CI/CD for EC2 with CodeDeploy
-

6. System Requirements

Functional Requirements

- User authentication and authorization
- Inventory CRUD and restock updating DB
- Orders create/update/delete with stock synchronization
- Supplier management
- Dashboard statistics and charts
- Data export and analytics querying

Non-Functional Requirements

- Secure access control and encrypted storage
 - Maintainable modular structure
 - Reliable errors in handling and logs
 - Scalable analytics querying via Athena
-

7. System Architecture

7.1 Overview (Figure 1)

Your system uses two flows:

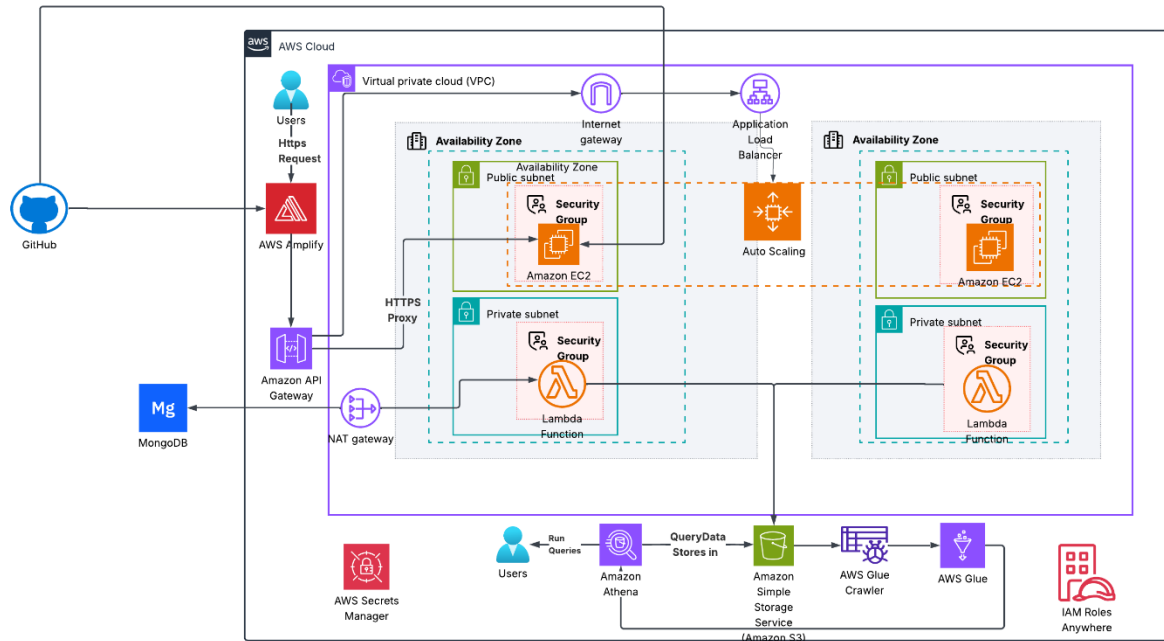
A) Operational Flow (App)

Users → AWS Amplify (React) → Backend API (EC2 Node/Express) → MongoDB Atlas

B) Analytics Flow (Export + Query)

MongoDB Atlas → NAT Gateway → Lambda Export (private subnet) → S3 exports

S3 exports → Glue Crawlers → Glue Data Catalog → Athena queries → S3 (athena-results)



Caption: “Overall architecture showing operational and analytics flows.”

7.2 VPC Network

- Lambda runs in private subnets
- NAT Gateway in public subnet provides stable egress
- Atlas allows NAT public IP /32

8. Implementation Details (Modules)

8.1 Authentication & RBAC

- JWT-based authentication
- Protected routes on backend and frontend
- Admin-only operations (e.g., delete order)

8.2 Inventory + Restock

- Restock dialog updates quantity in MongoDB through API endpoint
- Threshold/expiry support for monitoring

8.3 Orders Workflow

- Create order with item validation

- Confirm/cancel updates stock via controlled logic
- Delete order restricted to admin

8.4 Export Pipeline

- Lambda streams MongoDB documents → NDJSON → S3
 - Output structure:
 - exports/orders/...
 - exports/products/...
 - exports/users/...
 - exports/suppliers/...
-

9. Deployment Process

- Frontend deployed via Amplify from GitHub
 - Backend deployed on EC2
 - Lambda deployed via zip upload and environment variables
 - Glue/Athena configured to query exported datasets
-

10. Monitoring & Logging

- Lambda logs in CloudWatch
 - EC2 logs (if applicable)
 - Health check endpoint used for uptime validation
-

16. Results / Outputs

- Export files created in S3 per collection
 - Glue tables created and visible
 - Athena queries successful
 - Logs available for audit/debugging
-

17. Limitations

- NDJSON query performance not as optimal as Parquet
 - NAT gateway cost considerations
 - ALB/Auto Scaling planned but not implemented (if true)
-

19. Conclusion

The project achieves a secure operational pharmacy management system and a scalable analytics layer through AWS-managed services. Data is exported safely from MongoDB Atlas to S3 and becomes queryable via Glue and Athena, enabling reporting without impacting production workloads.

20. References (IEEE)

1. Amazon Web Services, “AWS Lambda Documentation,” 2025.
2. Amazon Web Services, “Amazon S3 Documentation,” 2025.
3. Amazon Web Services, “AWS Glue Documentation,” 2025.
4. Amazon Web Services, “Amazon Athena Documentation,” 2025.
5. Amazon Web Services, “Amazon Load Balancer,” 2025.
6. MongoDB, “MongoDB Atlas Documentation,” 2025.
7. Express.js, “Express Documentation,” 2025.
8. Mongoose, “Mongoose Documentation,” 2025.